STQA PBL VIVA VOCE COMPREHENSIVE PREPARATION GUIDE

Project: Student Attendance Management System (SAMS)

Department: Computer Science & Engineering, Bharati Vidyapeeth Deemed to be University

Team Members:

1. **Aditya Bafna** (Roll No. 04 | PRN: 2443110010)
2. **Adwai Singh** (Roll No. 07 | PRN: 2443110006)
3. **Aryan Singh** (Roll No. 16 | PRN: 2443110063)

Subject: Software Testing and Quality Assurance (STQA)

Supervisor / Guide: Prof. Dhanashree Waikar / Prof. Deepika Sharma

TABLE OF CONTENTS

1. [Team Role Allocation (Who Answers What)](#1-team-role-allocation)
2. [How the Project Was Built from Scratch](#2-how-the-project-was-built)
3. [Architecture & Technical Design Choices](#3-architecture--design-choices)
4. [Step-by-Step Live Viva Demonstration Script](#4-live-viva-demonstration-script)
5. [Top 25 Viva Questions & Model Answers](#5-top-25-viva-questions--answers)
   - [Category A: Project & Development Questions](#category-a-project--development)
   - [Category B: Mathematical & Business Logic Questions](#category-b-mathematics--business-logic)
   - [Category C: Core STQA Testing Questions](#category-c-core-stqa-testing)
   - [Category D: Boundary & Regression Questions](#category-d-boundary--regression)
6. [Key Definitions to Memorize](#6-key-definitions-to-memorize)

1. TEAM ROLE ALLOCATION (WHO ANSWERS WHAT)

When the examiner asks *"Who did what?"* or points to individual members:

Aditya Bafna (Roll 04) — System Architect & Frontend Lead

- **Primary Scope:** Architecture, Authentication, and Student Management.
- **Modules:** `index.html`, `dashboard.html`, `students.html`, `auth.js`, `students.js`, `style.css`.
- **What you should explain:**

- Why pure Vanilla JS and `localStorage` were chosen (zero-dependency, offline persistence).
- How login validation, password verification, and session state work.

- How student input validation (regex for roll numbers and names) and duplicate roll number prevention are implemented.

- How deleting a student cascades to remove their historical attendance records.

👤 Adwai Singh (Roll 07) — Core Business Logic & Calculation Lead

- **Primary Scope:** Attendance Processing Engine and Mathematical Reports.
- **Modules:** `attendance.html`, `reports.html`, `attendance.js`, `reports.js`.
- **What you should explain:**

- How date-indexed attendance matrices are structured and saved.

- How retroactive attendance updating works without duplicate records.

- The exact formula for Attendance Percentage: `(Present / Total) * 100`.

- How division-by-zero is safely handled when 0 classes are conducted.

- How the 75.00% eligibility threshold classifies students as `Eligible` or `Short Attendance`.

- How the search and filter functions operate in real time.

👤 Aryan Singh (Roll 16) — Quality Assurance & Test Automation Lead

- **Primary Scope:** STQA Test Suite, Test Documentation, and Verification.
- **Modules:** `tests.html`, `tests.js`, IEEE 829 Test Plan, Test Procedures (TP-01 to TP-10).
- **What you should explain:**

- The four levels of testing: Unit (UT01–UT15), Integration (IT01–IT06), Regression (RT01–RT05), and System (ST01–ST05).

- How Boundary Value Analysis (BVA) was applied to the 75% rule (0%, 74.99%, 75.00%, 75.01%, 100%).

- How Regression testing was conducted by establishing a baseline, making controlled code improvements, and verifying existing features.

- How the in-browser automated test runner measures execution latency (28.40 ms) and achieved a 100% pass rate.

2. HOW THE PROJECT WAS BUILT FROM SCRATCH

If the examiner asks: *"How did you build this project? Walk me through the development steps."*

Answer using these 5 systematic phases:

Phase 1: Requirements Analysis & Specification

- Identified the university requirement: Mandatory **75.00% attendance** for term exam eligibility.
- Formulated the academic requirements:

- Teacher authentication (admin / admin123).

- Student CRUD (Roll number, Name, Division) with strict uniqueness.

- Date-based daily attendance marking with Present/Absent toggles.

- Real-time mathematical calculation of attended vs conducted classes.

- Filtering by eligibility status and division.

- Selected a pure client-side stack (**HTML5, CSS3, Vanilla JS ES6+, localStorage**) to ensure zero external dependencies, fast execution, and reliable local operation.

Phase 2: Architecture & Data Modeling

- Adopted the **Model-View-Controller (MVC)** design pattern:

- **Model:** Three structured JSON stores in `localStorage`:

1. `sams_current_user` (session object)
2. `sams_students_data` (student roster array)
3. `sams_attendance_records` (date-keyed attendance dictionary)

- **View:** Clean semantic HTML5 pages styled with custom CSS3.

- **Controller:** Distinct JavaScript files handling logic and DOM manipulation.

Phase 3: Module Implementation

1. **Authentication (`auth.js`):** Implemented input trimming, empty-field checks, credential verification, and route guards (`initAuthProtection`).

2. **Student Management (`students.js`):** Built regex validation for roll numbers (`/^[A-Za-z0-9]{1,10}$/`) and names (`/^[A-Za-z\s.\-]{2,50}$/`), duplicate roll checks, and cascading deletions (`cascadeDeleteAttendance`).

3. **Attendance Engine (`attendance.js`):** Built date selection, Present/Absent status capture, batch actions, and non-destructive overwriting for retroactive edits.

4. **Reports & Math (`reports.js`):** Built pure mathematical functions with floating-point rounding: `Math.round(rawPct * 100) / 100` and the `>= 75.0%` decision rule.

Phase 4: STQA Test Design & Test Automation

- Authoring the **IEEE 829 Test Plan** (all 18 standard attributes).
- Authoring the **Test Procedure Document** (TP-01 to TP-10).
- Engineering an in-browser automated test harness (`tests.js` / `tests.html`):

- 15 Unit Tests (UT01–UT15) including BVA at 0%, 74.99%, 75.00%, 75.01%, 100%.

- 6 Integration Tests (IT01–IT06) verifying data pipelines between modules.

- 5 Regression Tests (RT01–RT05) checking baseline stability after code refactoring.

- 5 System Tests (ST01–ST05) testing complete end-to-end user workflows.

Phase 5: Verification & Quality Assurance

- Executed all 31 tests through the test harness.
- Verified 100% pass rate with zero runtime console errors.
- Captured real high-resolution screenshots for all views and test outputs.
- Compiled the complete 13-page academic PBL report in `.docx` and `.pdf`.

3. ARCHITECTURE & TECHNICAL DESIGN CHOICES

Why Vanilla JavaScript instead of React, Angular, or Vue?

- **Viva Answer:** *"Because this is an STQA academic project focusing on software testing principles. Using pure Vanilla JS gave us total transparency over memory state, pure deterministic functions, and native DOM manipulation without the overhead or unpredictability of virtual DOM diffing or third-party framework abstraction."*

Why localStorage instead of MySQL / MongoDB / Node.js?

- **Viva Answer:** *"Browser localStorage provides zero-configuration, instant client-side persistence. It allows the project to run immediately on any examiner's computer without installing web servers, database engines, or configuring port bindings, while still maintaining full relational state between page reloads."*

How do you prevent data corruption or orphaned records in localStorage?

- **Viva Answer:** *"Through relational integrity cascading in `students.js`. When a student is deleted via `deleteStudent(rollNo)`, the helper function `cascadeDeleteAttendance(rollNo)` iterates across all historical date keys in `sams_attendance_records` and deletes that student's status entry, preventing orphan attendance logs."*

4. LIVE VIVA DEMONSTRATION SCRIPT (HOW TO PRESENT)

Follow this exact 2-minute sequence when presenting to the external examiner:

```
Step 1: Open index.html
-----------------------
- Action: Click "Sign In" with blank fields.
- Show: Red validation alert: "Please enter both username and password."
- Action: Type "admin" and "admin123", click "Sign In".
- Show: Green success alert & instant redirect to dashboard.html.

Step 2: Dashboard Overview
--------------------------
- Show: Total Enrolled Students (6), Present Today (4), Absent (2).
- Show: Class Average Attendance (62.50%) and Students Below 75% (2).
- Explain: Metrics are dynamically aggregated in real time from localStorage.

Step 3: Student Management (students.html)
------------------------------------------
- Action: Enroll a new student (Roll: 107, Name: Aarav Mehta, Div: A).
- Show: Student added to roster.
- Action: Try adding Roll 107 again.
- Show: Error alert: "Roll number 107 already exists. Duplicates are not allowed."
- Explain: This demonstrates Equivalence Partitioning and Uniqueness constraints.

Step 4: Daily Attendance (attendance.html)
------------------------------------------
- Show: Date picker defaulting to current date.
- Action: Toggle Roll 101 to "Present", Roll 102 to "Absent", click "Save".
- Show: Toast notification confirming saved records.
- Action: Click on an older date chip (e.g., 2026-10-01).
- Show: Badge changes to "Editing Record (2026-10-01)" — demonstrates retroactive updating without
duplicate date keys.
```

```
Step 5: Cumulative Reports (reports.html)
----------------------------------------
- Show: Exact columns: Roll No, Name, Total Classes, Present, Absent, Attendance %, Status.
- Point to Rahul Sharma (Roll 101): 3 Present out of 4 Classes = Exactly 75.00% -> Green
"Eligible" badge.
- Point to Amit Kumar (Roll 103): 2 Present out of 4 Classes = 50.00% -> Red "Short Attendance"
badge.
- Action: Type "Rahul" in search box -> instant filtering. Select "Short Attendance" filter ->
displays only detained students.

Step 6: STQA Automated Test Runner (tests.html)  <-- THE MOST IMPORTANT STEP!
-------------------------------------------------------------------------------
- Action: Click "Testing Suite" in navigation.
- Action: Click "🚀 Run All 31 Tests".
- Show: Total: 31 | Passed: 31 | Failed: 0 | Pass Rate: 100.0% | Execution Time: ~28 ms.
- Click "Unit Tests (15)": Show UT11, UT12, UT13, UT14, UT15 (Boundary Value Analysis on 0%,
74.99%, 75.00%, 75.01%, 100%).
- Click "Integration Tests (6)": Show verified data flow across modules.
- Click "Regression Tests (5)": Show baseline stability after code refactoring.
- Click "System Tests (5)": Show complete end-to-end user journeys passing.
```

5. TOP 25 VIVA QUESTIONS & MODEL ANSWERS

CATEGORY A: PROJECT & DEVELOPMENT

Q1. What is the objective of the Student Attendance Management System?

Answer:

The objective is to provide an automated, zero-dependency classroom attendance tracking utility that replaces manual paper registers, eliminates calculation errors, computes cumulative attendance percentages, and audits examination eligibility against the university's mandatory 75% threshold in real time.

Q2. What technology stack did you use and why?

Answer:

We used HTML5 for semantic page layout, CSS3 for responsive styling and visual status badges, Vanilla JavaScript (ES6+) for modular controller logic and mathematical calculations, and HTML5 `localStorage` for offline persistence. We intentionally avoided heavy frameworks to maintain complete control over code execution and facilitate white-box and unit testing.

Q3. How does the data persist without a database like MySQL?

Answer:

We utilize the browser's native Web Storage API (`localStorage`), which stores key-value pairs persistently across browser sessions and computer restarts. We store three structured JSON objects: `sams_current_user` for user session, `sams_students_data` for student profiles, and `sams_attendance_records` for date-indexed attendance matrices.

Q4. What happens if the browser cache or localStorage is cleared?

Answer:

We implemented defensive fallback seeding. In both `students.js` and `attendance.js`, if `localStorage.getItem()` returns null or empty, the application automatically initializes a default realistic seed roster (Rolls 101–106 across Divisions A and B with 4 conducted sessions). This prevents application crashes and ensures seamless viva demonstrations.

Q5. How does the system handle security and authentication?

Answer:

We implement client-side session management via `auth.js`. When a teacher logs in with `admin` / `admin123`, a session token object is stored with a timestamp. Every protected page calls `initAuthProtection(true)`. If no valid session exists, the user is immediately redirected to `index.html`. Calling `logout()` destroys the session and redirects to login.

CATEGORY B: MATHEMATICS & BUSINESS LOGIC

Q6. What is the exact mathematical formula used for attendance percentage?

Answer:

For total conducted classes T where a student was evaluated, and attended classes P:

$$\text{Attendance Percentage} = \left(\frac{P}{T}\right) \times 100 \quad (\text{for } T > 0)$$

If $T = 0$, the function returns 0.00% and assigns the status `"No Classes"` to prevent division-by-zero errors.

Q7. How do you prevent floating-point inaccuracies in JavaScript?

Answer:

JavaScript uses IEEE 754 floating-point arithmetic, which can introduce precision errors (e.g., $0.1 + 0.2 = 0.30000000000000004$). To guarantee exact financial and academic rounding, our `calculateAttendanceMetrics()` function uses:

```
const rawPct = (safePresent / total) * 100;
const roundedPct = Math.round(rawPct * 100) / 100;
```

This guarantees exact two-decimal precision before formatting to a string with `.toFixed(2)`.

Q8. What is the university eligibility rule and how is it evaluated?

Answer:

Bharati Vidyapeeth mandates a minimum 75.00% attendance threshold to be eligible for semester-end examinations:

- If $\text{Percentage} \ge 75.00\% \implies \text{Status} = \text{"Eligible"}$ (Green badge)
- If $\text{Percentage} < 75.00\% \implies \text{Status} = \text{"Short Attendance"}$ (Red badge)

Q9. What happens if a teacher marks an invalid attendance status?

Answer:

In `attendance.js`, the `saveAttendanceForDate()` function iterates over every entry in the payload and validates it using `isValidAttendanceStatus()`. If any value other than `"Present"` or `"Absent"` is detected

(e.g., `"Late"`, `"HalfDay"`), the save operation is rejected, `localStorage` is not mutated, and an error message is returned. This is verified by Unit Test **UT10**.

Q10. What is cascading deletion and why is it necessary?

Answer:

Cascading deletion ensures relational data integrity. When a student is deleted from the roster (`sams_students_data`), their roll number might still exist in past date records inside `sams_attendance_records`. If left uncleaned, these orphaned records would corrupt class statistics. Our `cascadeDeleteAttendance(rollNo)` function iterates through all date keys and purges the deleted student's records from storage.

CATEGORY C: CORE STQA TESTING

Q11. What is the difference between Verification and Validation?

Answer:

- **Verification:** *"Are we building the product right?"* Evaluating work products (code, design, test plan) against specifications without necessarily executing code (static testing, code reviews, assertion checks).
- **Validation:** *"Are we building the right product?"* Evaluating the executing software to ensure it satisfies end-user operational needs and business requirements (dynamic testing, system testing, user acceptance).

Q12. What is the difference between a Test Case, a Test Plan, and a Test Procedure?

Answer:

- **Test Plan (IEEE 829):** A high-level management document outlining the scope, approach, resources, schedule, and pass/fail criteria for testing.
- **Test Case:** A set of preconditions, inputs, and expected results designed to verify a single specific requirement (e.g., UT01: Login with valid credentials).
- **Test Procedure:** A detailed, step-by-step operational instruction explaining exactly how a tester executes one or more test cases in sequence.

Q13. What is Unit Testing and how did you execute it?

Answer:

Unit testing verifies individual software units or functions in complete isolation. We authored 15 unit tests (UT01–UT15) in `tests.js` targeting:

1. `validateLogin()` (credentials and empty inputs)
2. `validateStudentInput()` (regex checks and duplicate roll numbers)
3. `calculateAttendanceMetrics()` (formula accuracy and boundary conditions)
4. `saveAttendanceForDate()` (valid and invalid status handling)
5. `deleteStudent()` (array manipulation)

Q14. What is Integration Testing and which interfaces did you verify?

Answer:

Integration testing evaluates the data communication and functional coupling between interconnected modules. We executed 6 scenarios (IT01–IT06):

- **IT01 (Auth → Dashboard):** Valid login initializes session and authorizes dashboard metric calculation.
- **IT02 (Student → Attendance):** Adding a student immediately surfaces their profile on the attendance marking sheet.
- **IT03 (Attendance → Storage):** Marking statuses persists the map correctly to `localStorage`.
- **IT04 (Storage → Reports):** Saved attendance immediately recalculates student totals in reports.
- **IT05 (Math → Eligibility):** Record counts flow into math formulas to produce correct eligibility statuses.
- **IT06 (Delete Cascading):** Deleting a student cascades removal across both roster and attendance records.

Q15. What is System Testing and how does it differ from Integration Testing?

Answer:

While integration testing focuses on interfaces between two or more modules, system testing tests the entire software end-to-end from the perspective of an actual user. We executed 5 comprehensive user journeys (ST01–ST05), such as logging in, enrolling a student, recording two days of attendance, and verifying that the final generated report correctly displays 100% attendance and eligibility.

Q16. What are Entry and Exit Criteria in your Test Plan?

Answer:

- **Entry Criteria:** Prerequisites required before testing begins (e.g., functional modules coded without syntax errors, test harness configured, test roster seeded).
- **Exit Criteria:** Conditions that must be satisfied to conclude testing (e.g., 100% of planned test cases executed, 100% pass rate on critical paths, zero unresolved high-severity bugs).

CATEGORY D: BOUNDARY VALUE ANALYSIS & REGRESSION

Q17. What is Boundary Value Analysis (BVA) and where did you apply it?

Answer:

Boundary Value Analysis is a Black-Box test design technique based on the principle that defects cluster at the boundaries of input domains rather than in the center. We applied BVA to the university's 75% attendance rule across 5 critical boundary points:

1. **0.00% (Minimum boundary - 0/10 classes):** Expected = `Short Attendance` (UT11)
2. **74.99% (Just below threshold - 7499/10000 classes):** Expected = `Short Attendance` (UT12)
3. **75.00% (On boundary - 3/4 classes):** Expected = `Eligible` (UT13)
4. **75.01% (Just above threshold - 7501/10000 classes):** Expected = `Eligible` (UT14)
5. **100.00% (Maximum boundary - 10/10 classes):** Expected = `Eligible` (UT15)

All 5 boundary tests passed with complete mathematical fidelity.

Q18. What is Equivalence Class Partitioning (ECP)?

Answer:

ECP divides input data into valid and invalid partitions where all members of a partition are expected to be treated equivalently:

- **Valid Partitions:** Percentage $\ge 75.0\%$ (Eligible), Percentage $< 75.0\%$ (Short Attendance).
- **Invalid Partitions:** Negative class counts, present classes exceeding total classes, student names with numbers/symbols, duplicate roll numbers.

Q19. What is Regression Testing and how did you perform it?

Answer:

Regression testing re-executes previously passing tests following code changes to ensure that enhancements or bug fixes have not broken existing functionality.

In our project:

1. We established a stable baseline where core login, student enrollment, and attendance marking were verified.

2. We introduced controlled refactoring: string sanitization in `auth.js`, hyphenated name regex support in `students.js`, division-by-zero protection in `reports.js`, and two-decimal precision rounding.

3. We executed our 5-test regression suite (RT01–RT05) against the modified codebase. All 5 tests passed, confirming zero regressions.

Q20. How did your automated test harness measure test execution time?

Answer:

We utilized the browser's high-resolution `performance.now()` API in `tests.js`. By capturing timestamps immediately before and after executing test assertions, the harness calculates execution duration in milliseconds:

```
const startTime = performance.now();
// ... execute test assertion ...
const duration = (performance.now() - startTime).toFixed(2);
```

The entire 31-test suite executes in approximately **28.40 milliseconds**.

Q21. What is White-Box vs. Black-Box testing in your project?

Answer:

- **Black-Box Testing:** Testing without knowledge of internal code structures. Applied during System Testing (ST01–ST05) and Boundary Value Analysis on the 75% rule by providing inputs and evaluating UI outputs.
- **White-Box Testing:** Testing with full knowledge of internal code logic and branches. Applied during Unit Testing (UT01–UT15) to achieve 100% statement and condition coverage across `validateStudentInput()`, error branches, and division-by-zero checks.

Q22. How did you validate student roll numbers and names?

Answer:

Using JavaScript Regular Expressions:

- **Roll Number:** `/^[A-Za-z0-9]{1,10}$/` — ensures 1 to 10 alphanumeric characters without special symbols or spaces.
- **Student Name:** `/^[A-Za-z\s.\-]{2,50}$/` — ensures 2 to 50 characters containing only letters, spaces, dots, and hyphens (rejecting numbers and symbols like `@#$%`).
- **Uniqueness Check:** `existingStudents.some(s => s.rollNo.toLowerCase() === cleanRoll.toLowerCase())` — rejects duplicate entries regardless of letter casing.

Q23. Why is division-by-zero an important edge case in this system?

Answer:

If a student is enrolled but zero classes have been conducted yet ($T = 0$), calculating $(0 / 0) \times 100$ in JavaScript results in `NaN` (Not a Number). Displaying `NaN%` on an academic report is a critical UI and mathematical defect. Our calculation engine checks `if (total === 0)` first and returns 0.00% with the status `"No Classes"`, eliminating NaN bugs. This is verified by Regression Test **RT04**.

Q24. What are the key deliverables of your STQA PBL project?

Answer:

1. Fully working, zero-dependency web application ([`index.html`](file:///C:/Users/ZFXMIKEY/.gemini/antigravity/scratch/Student-Attendance-System/index.html)).

2. STQA automated test harness executing 31 test cases ([`tests.html`](file:///C:/Users/ZFXMIKEY/.gemini/antigravity/scratch/Student-Attendance-System/tests.html)).

3. IEEE 829 Software Test Plan Document.

4. Software Test Procedure Document (TP-01 to TP-10).

5. Formal 13-page academic PBL Report in `.docx` and `.pdf` formats conforming to university guidelines.

6. Empirical test output logs in JSON format ([`test_execution_results.json`](file:///C:/Users/ZFXMIKEY/.gemini/antigravity/scratch/Student-Attendance-System/test_execution_results.json)).

Q25. What are the future enhancements for this application?

Answer:

1. **Role-Based Access Control (RBAC):** Implementing distinct views for HODs (department-wide analytics), Teachers (marking attendance), and Students (viewing personal attendance).

2. **Automated Hardware Integration:** Incorporating biometric fingerprint scanners or RFID smart cards for contact-free attendance capture.

3. **Automated Notification Gateway:** Integrating SMS and email APIs to alert parents automatically when a student's attendance drops below 75%.

6. KEY DEFINITIONS TO MEMORIZE (CHEAT SHEET)

STQA	Software Testing and Quality Assurance: A systematic set of activities to ensure software adheres to defined requirements and quality standards.
BVA	Boundary Value Analysis: A black-box test design technique that tests values at the boundaries of equivalence partitions (min, just below, on, just above, max).
ECP	Equivalence Class Partitioning: Dividing input data into partitions where all inputs in a group produce equivalent system behaviors.
Regression Testing	Re-testing unchanged parts of an application after code modifications to confirm that existing functionality has not been degraded or broken.
Integration Testing	Testing the interactions and data pipelines between two or more integrated software units or modules.
System Testing	Testing a completely integrated system end-to-end to verify that it meets all specified functional and user requirements.
IEEE 829	The international IEEE standard defining the format and content of Software Test Documentation (Test Plan, Test Design, Test Procedures, Test Logs).
Cascading Deletion	A database/storage constraint where deleting a parent record (student) automatically deletes all associated child records (attendance logs) to prevent orphaned data.